PROJECT BRIEF

Proofed

Integrated ASR + post-processing model, from speech straight to clean, proofed text

Daniel Rosehill · 2026-10-24

Defined by	Daniel Rosehill, voice note recorded 2026-10-24 (24:40 long)
Prepared by	Claude Code · Opus 5.5 (claude-opus-5-5)
Generated	2026-10-24 14:20:18 +03:00
Transcription	AssemblyAI universal (transcript fbc93846-95b4-4d0b-8121-e45a285ccebb), lightly edited by Claude
Repository	github.com/danielrosehill/Proofed @ 3806576

Summary

Proofed fine-tunes a stock, phone-sized speech-recognition model (Parakeet TDT 0.6B v2) only on cleanup tasks, so it writes clean text in one pass: no fillers, repeats or false starts, numbers as digits, self-corrections applied, asides left out, paragraphs, and whispered speech handled. Domain vocabulary is out of scope. The first phase is research: Parakeet's paragraph token, prior art, candidate base models and feasibility.

Contents

1 Overview 3
- 1.1 What it is 3
- 1.2 The problem 3
- 1.3 The objective 3
- 1.4 Scope 3
- 1.5 Approach 3
- 1.6 Background 3
- 1.7 Open questions from the note 4

2 Development plan 4
- 2.1 What it is 4
- 2.2 Why 4
- 2.3 Scope 4
- 2.4 Approach 5
- 2.5 Phases and exit tests 6
- 2.6 Open questions (need Daniel) 6
- 2.7 Risks 6

3 The voice note, lightly edited 7
- 3.1 Background: voice as the way in (00:00) 7
- 3.2 The basic contract of ASR (01:58) 7
- 3.3 From post-processing prompts to audio-native models (02:52) 7

3.4 Why that isn't the whole answer: size (04:55) 7
3.5 Dedicated post-processing models (05:53) 8
3.6 What I've been fine-tuning (06:11) 8
3.7 One model across devices (07:52) 8
3.8 How the task list grew (08:37) 8
3.9 Combining domain fine-tunes didn't hurt (10:58) 9
3.10 The idea: Proofed (12:34) 9
3.11 Layering (13:55) 9
3.12 Where the line is (15:06) 9
3.13 Paragraph inference (16:19) 10
3.14 The experimental one: non-directed speech (17:04) 10
3.15 Keep tasks narrow and modular (17:54) 10
3.16 The objective (19:27) 10
3.17 The prototype (20:25) 10
3.18 Whispered sentences (21:03) 10
3.19 Data and prior art (22:00) 11
3.20 Stages (22:47) 11
3.21 Open question: English-only? (23:39) 11
3.22 Correction: domain vocab is out of scope (23:54) 11
3.23 Corrections 11

1 Overview

1.1 What it is

Proofed is a speech-recognition model that writes clean text directly. It starts from a stock, phone-sized ASR model (NVIDIA Parakeet TDT 0.6B v2) and fine-tunes it only on post-processing tasks, so the transcript comes out already proofed, in a single forward pass, with no LLM step afterwards (12:34, 19:27).

1.2 The problem

The basic contract of ASR is narrow: render sounds as words. Everything beyond that (punctuation, sentence boundaries, removing “um”) is an addition someone chose to build (01:58). Today those additions come from a second model: a general LLM with instructions, or a dedicated text post-processor such as Superwhisper’s S1-mini (05:53). Audio-native multimodal models can do any transformation but are far too big for a phone and unusable for dictation even on a desktop GPU with 12 GB of VRAM (04:55). Regex cleanup after inference works for some of it; the aim is to fix it at source (16:03).

1.3 The objective

Find out to what extent a stock ASR model can be fine-tuned to bake in a set of post-processing transformations (12:34). Same result for the user as ASR plus cleanup, from one small model. Published on Hugging Face as an experimental prototype (20:25).

1.4 Scope

In: transformations that are right for every user every time (15:17):

- removing fillers (“um”, “uh”), repeated words, false starts and unfinished sentences
- numbers written as digits
- applying self-corrections and editing instructions (“Haifa, I mean Tiberias”; “scratch that”)
- paragraph breaks
- leaving out speech aimed at someone else in the room (the experimental task)
- whispered speech, so dictation works when others are asleep (21:03)

Out: any domain vocabulary, including code-switching and tech terms (23:54), and anything use-specific, such as formatting every output as an email (15:17).

1.5 Approach

Take Daniel’s existing “kitchen sink” recordings, already stratified by task type, keep only the post-processing tasks, and tier them as core, ambitious and experimental (20:25). Define each task narrowly so each can be evaluated alone, since some may not train well at small scale (17:54). Later, test a layered approach: fine-tune Proofed again with a domain-vocabulary layer and compare with a one-shot model (13:55).

1.6 Background

Proofed grows out of Daniel’s Parakeet and Nemotron fine-tunes for English-Hebrew code-switching and tech vocabulary (06:11). While adding number rendering and disfluency removal to that dataset, he noticed the work had crossed from domain-vocabulary fine-tuning into post-processing (12:34). An earlier finding encouraged him: combining two domain fine-tunes in one “kitchen sink” dataset did not degrade either (10:58).

1.7 Open questions from the note

- Does Parakeet have a token for a blank line? If not, paragraphs need a different base model or a decoder change (16:19).
- Can one model hold both an ASR pass and a post-processing pass internally (19:27)?
- Can the model be English-only (23:39)? With domain vocabulary out of scope, probably yes.
- Which existing attempts at a unified ASR and post-processing model are there (22:00)?

2 Development plan

Status 2026-10-24: planning. No data split, no training run, no model yet. Source: voice note 2026-10-24 (cited as `vn mm:ss`). Where this plan and the note disagree, the note wins; record the decision here.

Decided = said by Daniel in the note. **Inferred** = this plan's reading, to confirm.

2.1 What it is

An ASR fine-tune that writes clean text directly: a stock ASR base model (Parakeet first) trained only on post-processing tasks, so that the transcript comes out without disfluencies, with numbers as digits, without asides to other people, and (if the base model allows) with paragraphs. One forward pass, no second LLM step. (Decided, vn 12:34, 19:27.)

It is a research prototype: the question is *to what extent a stock ASR model can be fine-tuned to bake in a series of post-processing transformations* (vn 12:34). Published on Hugging Face (vn 20:25).

2.2 Why

- Audio-native LLMs (Gemini with a system prompt) can do any transformation, but they are too big for phones and unusable for dictation even on a 12 GB desktop GPU (vn 04:55).
- Dedicated text post-processors (S1-mini by Superwhisper) still need a second model (vn 05:53). Daniel's own text-side attempts: `danielrosehill/ASR-Post-Processor-Model` (planning, S1-mini LoRA) and `danielrosehill/ASR-Paragraph-Breaks` (Gemma 3 270M, break F1 0.78, released 2026-10-21).
- Regex post-inference cleanup works for some of this; the aim is to fix it at source (vn 16:03).
- The Hebrish kitchen-sink run showed that combining training domains did not degrade either (vn 10:58). Inferred: that is some evidence task subsets can be stacked.

2.3 Scope

In (decided): post-processing transformations that are right for every user, every time (vn 15:17). Narrow, modular tasks, each measurable on its own, because some may not train at small scale (vn 17:54).

Out (decided): all domain vocabulary, including code-switching, tech vocab, names (vn 23:54). Anything use-specific, such as formatting output as an email (vn 15:17).

Tiers. Daniel names core, ambitious and experimental tiers (vn 20:25) but assigns only some tasks. Proposed mapping onto the existing task categories (`audio-dictation/Hebrish-App/data/training-categories.json`, 2026-10-21):

Tier	Category id	Task	Basis
core	`disfluency`	drop uh / um / eh	decided, vn 09:55, 15:17
core	`repetitions`	drop restart repeats (“orange, orange juice”)	decided, vn 17:54
core	`false-starts`	drop abandoned starts and unfinished sentences	decided, vn 17:54
core	`numerics`	numbers, dates, units in digits	decided, vn 09:55
core	`currency`	spoken amounts as typed (50 NIS)	inferred: general formatting
ambitious	`self-corrections`	“Haifa, I mean Tiberias” → Tiberias; “scratch that”	inferred tier; task decided, vn 18:32
ambitious	`paragraphs`	paragraph breaks	decided task, vn 16:19; tier inferred, blocked by tokenizer (below)
experimental	`asides`	drop speech aimed at someone else	decided, vn 17:04
robustness	`whispered`	whispered speech	decided, must be included, vn 21:03
open	`web-addresses`, `spelled-out`, `acronyms`, `expanded-acronyms`	written-form conventions	unclear whether “post-processing” or “domain”; ask
out	`singing`, `weird-voices`, `name-cipher`	other experiments	inferred out: not mentioned

Plus plain-English replay (`general`, LibriSpeech-PC) so the base does not forget; ratio per `ai-models/ASR-Dataset-Blending` (start ~10%).

2.4 Approach

1. **Base model:** `nvidia/parakeet-tdt-0.6b-v2` (English, punctuated and cased). Daniel prefers to stay on Parakeet (vn 22:47). Survey alternatives anyway (task in `tasks.md`).
2. **Data:** from the kitchen-sink recordings (`danielrosehill/Read-Aloud-Voice-Sentences` and the Hebrish blends, e.g. `Hebrish/Parakeet-V2-Blend-All`), keep only the post-processing categories above, stratified by category. Same term-disjoint train/test split as the Hebrish repos. Measure minutes per category first: Daniel expects some need more recording (vn 22:00).
3. **Training:** NeMo fine-tune, as in `audio-dictation/English-Hebrew-Codeswitching-Parakeet-Finetune` (Modal GPU).
4. **Evaluation:** per category, against the clean target: task success (filler gone, digits right, aside removed, correction applied) plus WER on the rest of the sentence, and WER on `general` and

LibriSpeech-PC for regression. Compare with stock Parakeet, and stock Parakeet + regex + `ASR-Paragraph-Breaks` as the two-model baseline.

5. **Layering (later):** fine-tune Proofed again with a domain-vocab layer (tech, code-switching) and compare with the one-shot kitchen-sink model (vn 13:55).

2.5 Phases and exit tests

Phase	Work	Exit test
0. Brief	this repo, cleaned transcript, plan, tasks	done 2026-10-24
1. Research	prior art (unified ASR + post-processing models), ASR models with a paragraph/ newline token, feasibility of training these tasks into an ASR decoder, candidate base models	`docs/research.md` with sources, dates, and a confirmed/inferred status per claim
2. Data audit	minutes and takes per in-scope category; gaps	table in `docs/data.md`; every core category ≥ the minimum Daniel sets
3. Spec	final task list and tiers, targets, metrics, blend ratios	`docs/spec.md` agreed
4. Prototype	first fine-tune on core + whispered	per-category eval vs stock Parakeet; no regression over 1 pt WER on `general`
5. Extend	ambitious and experimental tasks one at a time	each task's eval, kept or dropped with a reason
6. Publish	HF model, card, eval table	model card live

2.6 Open questions (need Daniel)

1. **English-only base?** Raised at vn 23:39. With domain vocab out of scope (vn 23:54), inferred answer: yes, the English-only Parakeet v2 is fine; code-switching comes back only in a later layer, and that layer's base question can wait. Confirm.
2. **Paragraphs.** Parakeet's tokenizer may have no newline token (vn 16:19, "if I'm not mistaken"); to verify in phase 1. If not: add a token and resize the decoder, change base model, or leave paragraphs to `ASR-Paragraph-Breaks`?
3. **Tiers.** Confirm the table above, especially the "open" row.
4. **One model or two passes inside one?** vn 19:27 asks whether one model can hold an ASR pass and a post-processing pass. Treated as a research question in phase 1, not a design commitment.
5. **Recording budget.** How many more minutes per thin category is Daniel willing to record?

2.7 Risks

- Tasks such as asides and self-corrections need context the ASR decoder may not model well at this size; they may not train (Daniel expects some won't, vn 17:54).
- Cleaning in the decoder can over-delete real words; the eval must measure WER on the kept text, not only task success.
- Read-aloud disfluency data is acted, not natural; it may not transfer to real dictation.

3 The voice note, lightly edited

Voice note recorded 2026-10-24 by Daniel Rosehill (24:40). Cleaned from `transcript-raw.txt` (AssemblyAI universal, unedited). Fillers, false starts and repetitions are removed and obvious mis-hearings fixed (listed under Corrections at the end). The words, order and points are the speaker's. Timestamps are `mm:ss` into `voice-note.mp3`.

3.1 Background: voice as the way in (00:00)

This voice note was recorded on 24 October 2026. It's context for a project that builds on what I've been working on for the past week, and really on a couple of preceding years of using voice dictation all day, every day, across different devices, as my preferred way of entering text into a computer: trying out my own fine-tunes, trying out different apps, seeing what works and what breaks, and seeing how the available models have changed and are changing. That goes from the amazing Whisper, an outstanding model, to the directions people have been taking for that final push in ASR and voice dictation utility: from "this is very impressive, very accurate, but I still need to put effort into this before I send it [unclear: "to the ladder"]" to something you don't have to touch.

I think this is a very exciting area right now, because what I think is essential is the action taken on the back of voice notes. If it stops at the transcription, a huge amount of the potential utility of dictation and voice as an interface for getting things done is lost, maybe even most of it. With easy-to-use connectors for email, calendar, contacts, you name it, available in apps like ChatGPT, the complication and bugginess of MCP is no longer something that most people, or anyone for that matter, has to wade through just to get these things done.

3.2 The basic contract of ASR (01:58)

The basic contract of ASR is extremely limited in scope: taking what people say, phonetics, sounds, and rendering that into words. What's underappreciated is that everything bolted onto that very bare level of scope is a decision and an addition. It's work that someone undertook to go beyond the basic contract. Someone said: hang on, we need sentences, we need punctuation, we need to detect sentence boundaries, and so on.

3.3 From post-processing prompts to audio-native models (02:52)

There have been a couple of changes I've been looking at for my own tools. Over the course of building these out, I developed a bank of what I call text transformation prompts. Initially that was the classic post-processing step for ASR plus LLM: the second step gets the ASR text back and says, restructure this text as an email, a formal email, a casual email, a shopping list, a task list. When I was drafting these, I realised the number of permutations was enormous, since you can build them one on top of the other.

I did create an app for my own use that did exactly that. Under the hood it was prompt concatenation. I had boxes for "make this more formal", "less formal", "polite business email"; some of them instructing the output format, some of them instructing the style. When the multimodal models started appearing, the Gemini models that could process audio tokens and take a system prompt, it struck me that this is gigantic for dictation, because I can create basically any instruction, even "translate this into formal Hebrew". In other words, I can mop up even translation tools.

My thought at that point was: what's the point of having ASR and cleanup models if you have these? I've been thinking about that a lot since. Is that the future?

3.4 Why that isn't the whole answer: size (04:55)

One of the negatives of that line of prediction is that, to the best of my knowledge, there aren't very many small models that can do audio. On-device ASR has quickly matured, through excellent models

from Moonshine and NVIDIA. The multimodal audio models are still just too big for most people. I don't even know what phone could run them; they're not practical for the most part. Even on the desktop with 12 GB of VRAM, I haven't found one that runs in any way that's usable for dictation, which is a separate concern.

I'm tracing the full evolution of the field as I see it, to explain where I see this idea fitting in.

3.5 Dedicated post-processing models (05:53)

Then there's a model called S1-mini by Superwhisper, which is a dedicated post-processing model. In other words, instead of taking a general-purpose LLM and adding your own instructions to an instruction-tuned model, it's something actually trained to expect ASR text, with a defined system prompt honed on this specific task. That's a good idea.

3.6 What I've been fine-tuning (06:11)

In the last couple of weeks I've been fine-tuning Parakeet and Nemotron on a code-switching task. I live in Israel, and when I began using Whisper very ardently, one of the first things that jumped out at me was that it was going to fail on all the Hebrewisms, as it were: "I'm going to the shuk and going to Bituach Leumi after this." So my first fine-tune for Whisper was code-switching.

Then, again just because I'm using these things day in, day out, I noticed that tech vocab and new brand names are a problem. That makes sense, because they're still fairly niche, and beyond being niche they change very fast. I might begin talking about S1-mini from Superwhisper a lot, or ROCm, as in ROCm for the AMD stuff.

For this round of fine-tuning I moved from Whisper to Parakeet, because I found that stock Parakeet was far, far superior on my phone, which is what I've happened to be using for the past few months rather than my desktop.

3.7 One model across devices (07:52)

I'm personally in favour of using one model across different devices. You might have a model that's the best choice for GPU acceleration on one machine. But this is my philosophy: if you're working towards extremely high accuracy and exactly the type of output you want, there's more to be said for knowing exactly the limits, the pros and the strengths of one model than for using a couple of them. It gets hard, it gets confusing: which model doesn't handle the disfluencies, which does. So I'm going all in on Parakeet and the NVIDIA models generally.

3.8 How the task list grew (08:37)

For the fine-tune I started training last week, after I moved over, I didn't even consider using my previous Whisper dataset, because I thought it was a bit old. So I started recording new code-switched sentences. And while doing that I thought: if I'm going to sit down to do this… It's worth it when you start seeing the evals move in the right direction, but it's kind of weird to be sitting in bed reading these random sentences, and it's tedious.

So I thought, if I'm going through this process again, I may as well think of everything I want: everything that's wrong with this model as I see it, or everything it does poorly. After code-switching I added recognition for tech vocab, and then I asked what else would be nice for this model to do, and began drawing up a list as I transcribed with it. These are things I've just observed; it's not as strong as Whisper in some ways, maybe because it's a smaller model.

One was number rendering. The classic example: "the year is 2026" could be rendered in spelled-out numbers, but no one writes a date that way, so it should be 2026 in numerals. Then just disfluency removal: Parakeet very faithfully renders all the ums and ahs.

As I built up these tasks and datasets and trained on them, I realised I had crossed a juncture, or a division.

3.9 Combining domain fine-tunes didn't hurt (10:58)

My first experiment was to see whether two sets of fine-tuned vocab would degrade each other's accuracy, which my evals said they didn't. That was very encouraging, because I try to balance my desire to open source with the time I have to dedicate to a project. In an ideal world I'd maintain a tech one, because that could be useful for a lot of people, and Parakeet fine-tuned for code-switching plus tech for my own use. But in the real world, to the extent that I can afford to spend time on my open-source projects, it's biased towards the tools I'm actually going to use and benefit from most myself, which is the all-in-one.

I called it the kitchen-sink training dataset: tech plus Hebrew. That was a surprising result. I assumed, and I'm not sure why, it just seemed common sense, that if you distribute the weights you're increasing recognition for across tech vocab and Hebrew, the jack of all trades is going to be master of none. And even if the eval had shown a 5% decrease in accuracy on both domain datasets, I would have said that's a good trade-off, because the most useful model for me is the one that does code-switching and tech without overfitting on either niche domain.

3.10 The idea: Proofed (12:34)

Coming back to the thought I was finishing earlier: when I began expanding the dataset, because who wants to do this multiple times, I realised that the first two datasets are training on the classic ASR fine-tuning task of domain vocabulary. The second group, which I'm building out now, crossed over once I added disfluency removal and number rendition: we're now integrating a post-processing step into the model instead of doing it separately.

So what I want to train with this model, which I'm calling Proofed (I couldn't think of a better name right now, but it's a decent one), is an ASR model trained only on a base ASR model plus cleanup. I'm not sure this is actually the fine-tuning methodology that makes sense, but it would nevertheless be very worthwhile, just to see how well this works in isolation. This is the isolated model, to test to what extent a stock ASR model can be fine-tuned to bake in a series of post-processing transformations.

3.11 Layering (13:55)

Then, can we take a layering approach and fine-tune the fine-tune to add the second layer of needs, like the tech vocab? In other words, you take a model like Parakeet; Proofed is Parakeet plus fine-tuning for post-processing and formatting; and then you build on top of it. It's a higher workload from a training perspective, since the easiest way to get to any result is one dataset and one training run, but it could be a very elegant way to do it.

What I want to do for this one, maybe not for my own use but as an experiment, is throw in as many of the datasets I've been creating specifically for post-processing as I can. I've deliberately created a mixture, starting with what I'd call the core ones.

3.12 Where the line is (15:06)

There is a limit, a definite line you don't want to cross. If you're going to bake in deterministic post-processing, you want it to stay general. (And maybe this is where it could pair with a second pass.) If, for argument's sake, Proofed were trained to format every output as an email, it would be useless for anyone who wants it for any other purpose. So the set of post-processing transformations I'm trying to train on are the ones that make sense for everyone. For my use, I can't think of a single time I've wanted disfluencies like "um" to appear in a transcript. They're always unwanted, and that's the classic example of the type of thing I want to bake in here.

Those can be handled at the regex level, meaning post-inference. But the idea here is to make that unnecessary in the first place, to tackle it at source.

3.13 Paragraph inference (16:19)

Another one I feel strongly about is paragraph inference. I find it's very overlooked, in ASR and even in post-processing: sentence boundary detection is one thing, but adding paragraph breaks is another. This is a bit of a problem with Parakeet, because if I'm not mistaken the core model doesn't have a token for a blank line, which makes training for paragraph inference impossible in the raw model. So I might need to find a different base model, or add that into the decoder.

3.14 The experimental one: non-directed speech (17:04)

Those are the standard ones on my list. The experimental one, as I call it, is non-directed speech, which is kind of a reasoning task. For example, you're recording and you say: "So the API documents are here now... yeah, just leave it by the door." The idea is to infer that "just leave it by the door" wasn't meant for the transcription: the speaker was talking to somebody else, and it shouldn't be included. That's an edge case: if it worked, it might be very useful; if it didn't, not so much.

3.15 Keep tasks narrow and modular (17:54)

I think it makes sense to define these tasks very narrowly and modularly, because in seeing what actually works, it could be that certain tasks just can't be trained reliably at a small scale and some can.

The other common disfluency is repeated words: "I'm putting the orange, orange juice in the fridge." You repeat words when you're distracted. False starts are another: "And going to the... I'm going to the cinema." Another example is sentences you never finish: "I think I had an idea... I want to go to the movies now."

Another is inferred editing instructions. A self-correction: "I need to go to Haifa today. I mean Tiberias." Whereas an editing instruction might be: "I need to go to Jerusalem. Scratch that, Tel Aviv." These have been handled differently by different approaches and models in ASR and speech, and the question is really scope: which of these makes sense?

3.16 The objective (19:27)

What I'd like for my own use: rather than a huge multimodal model capable of processing everything, and therefore very heavy, take a pure ASR model and adapt it so that in a single forward pass it not only does the phonetic rendering but also reasons and produces text that follows these instructions. I don't know if this is architecturally possible: one model that encapsulates two passes, the first pure ASR, the second post-processing. Same result for the user; internally, two different processes going on. I'm not sure, but that's the idea.

3.17 The prototype (20:25)

For the prototype on Hugging Face: take my training dataset, which is basically the kitchen sink, everything I've recorded, stratified by all these types. Isolate only the post-processing tasks, and within those, isolate the ones I've described here as core, ambitious and experimental.

3.18 Whispered sentences (21:03)

There's one in there I've called whispered sentences, and it's an important one for me. Here's the use case. Since I became a voice-first user, sometimes I'm in bed and my wife is saying "stop talking to your phone". I remember asking ChatGPT way back whether Whisper could transcribe whispered speech. Then I just tried it, and it was quite good. I thought, this is actually super useful: maybe you have to write something discreetly, maybe you're trying not to wake the kids, or the wife, or the husband. Why not include an actual small corpus of whispered sentences in the training subset, so

there's explicit training data on whispered speech? That's why I created a whispered dataset, and I'd very much like to have it included for that reason.

3.19 Data and prior art (22:00)

The datasets I have at the moment might need to be beefed up for all of these tasks, paragraph inference and the others. I'd also like a survey of prior art, and parallel art as well: ASR models that support a paragraph break as a token, and anything else. I'd be surprised if there weren't other attempts out there to create a unified ASR post-processing model. (I don't even like that name very much, but it's accurate to the lineage, and that's the objective.)

3.20 Stages (22:47)

There are a few stages. First, setting up the repository with a cleaned-up version of this transcript and a spec from it. Then research on existing attempts, research on feasibility, and research on candidate ASR models that might be better fits than Parakeet, although I'd be reluctant not to use it, because I've become very fond of it.

3.21 Open question: English-only? (23:39)

The other question I have is within the Parakeet models. This would be an English model. If I bake in my code-switching support, I presume it can still be a model that explicitly supports only English? That's an open question: whether it has to be multilingual if code-switching is part of it.

3.22 Correction: domain vocab is out of scope (23:54)

Actually, I'll correct myself and end by clarifying: the idea for this one is to isolate, in the proof of concept at least, the post-processing elements, so that future trainings of a model that integrates niche domain vocab (say my tech model) build on an established base that does the post-processing reliably. So any domain vocab is out of scope for this one. This is the experimental prototype, just for the post-processing tasks combined.

3.23 Corrections

Mishearings fixed against `transcript-raw.txt`:

Raw	Clean	Note
“into a compute out my own fine tunes”	“into a computer: trying out my own fine-tunes”	
“send it to the ladder”	kept, marked [unclear]	possibly “send it off” or “send it to the LLM”
“Parakeet Nematron”	“Parakeet and Nemotron”	both lines are being fine-tuned (Hebrish Shakshuka is the Nemotron one)
“Beto Akhlaomi”	“Bituach Leumi”	
“Whisperer”	“Whisper”	
“ROCM”	“ROCm”	
“evalues”	“evals”	
“tac flow cap”	“tech vocab”	
“the m’s”	“ahs”	
“full starts”	“false starts”	
“adding instruction”	“editing instruction”	
“Tiberius”	“Tiberias”	
“Candidus ASR models”	“candidate ASR models”	
“S1 Mini by Superwhisper”	“S1-mini by Superwhisper”	
“experimentary”	“experimental”	the speaker’s own coinage; same meaning
“Moonshine and Nvidia”	“Moonshine and NVIDIA”	

At 23:54 the speaker revises the scope: domain vocabulary, including code-switching, is out of scope for Proofed. The question at 23:39 is left as spoken; `docs/development-plan.md` records the resulting decision.